Обучение без учителя

Введение в обучение без учителя. Unsupervised Learning.

Ранее мы рассматривали задачи обучения с учителем, где:

- Была дана выборка $\mathbb{X}$ с целевыми переменными $\mathbb{Y}$;
- Необходимо построить функцию $a : \mathbb{X} \to \mathbb{Y}$ такую, что $a(x) \approx y$ на каждом ожидаемом объекте;
- Требование выше разумно для задачи регрессии ($y \in \mathbb{R}$); для задачи классификации ($y \in \{0, 1\}$) стоит требовать точного попадания;
- С помощью bias-variance tradeoff можно показать, что слишком лёгкие модели склонны недообучаться, а слишком сложные — переобучаться. На это также влияет качество и количество самих данных: на маленьком количестве данных не получится хорошо обучить чересчур сложную модель;
- В таких задачах можно конкретно установить функцию потерь (то, что требуется минимизировать), а также задача была, в целом, крайне понятной и легко интерпретируемой.

> **Замечание**
>
> Также нам удалось установить, что ансамблевые методы (Random Forest, градиентные бустинги) могут компенсировать bias-variance tradeoff на уровне архитектуры, что делает их методами, которые способны крайне эффективно справляться с подобными задачами.

Но встречаются задачи, в которых целевой переменной просто нет. К таким задачам относятся:

- Детекция аномалий;
- Кластеризация;
- Визуализация данных / понижение размерности.

В рамках данной лекции мы познакомимся с задачами кластеризации и понижения размерности и узнаем, как и чем их возможно решать.

Формулировка задачи

Прежде чем идти далее, давайте более явно сформулируем идею обучения без учителя:

Определение 1 (Unsupervised Learning) Обучение без учителя (Unsupervised Learning) предполагает, что в данных отсутствует целевая переменная y. Такие задачи ограничиваются первичным анализом данных с целью выявить какие-то интересные закономерности в них.

Возникает сразу множество разумных вопросов: как в теории теперь возможно валидировать качество модели, если ранее мы делали это на основе y, которого сейчас нет?[1] Что теперь с градиентным спуском? Да и вообще, как такую задачу возможно решать?

[1] Вспомните основные функции потерь, которые мы применяли: MSE, MAE, RMSE…

Задача кластеризации (Clustering)

Чтобы ответить на эти вопросы, давайте сразу рассмотрим одну из таких задач — задачу кластеризации.

Определение 2 (Clustering) Кластер — это группа объектов, объединённых по определённым признакам, причём в рамках этого признака объекты похожи друг на друга в одном кластере, а объекты из разных кластеров не похожи друг на друга. Кластеризация — процесс разбиения множества объектов на N кластеров.

⚠ Предупреждение

Идейно кластеризация похожа на задачу классификации, с двумя «небольшими» (в сущности, невероятно важными) но:

1. Согласно Определение 1, у нас по-прежнему нет целевых меток y, которые ранее отвечали за метку класса;
2. Задача кластеризации исследует природу объектов: то, как они лежат относительно друг друга, и на какие группы можно их разбить в зависимости именно от этого, а не только в зависимости от значения y.

Задачу кластеризации можно визуализировать следующим образом:

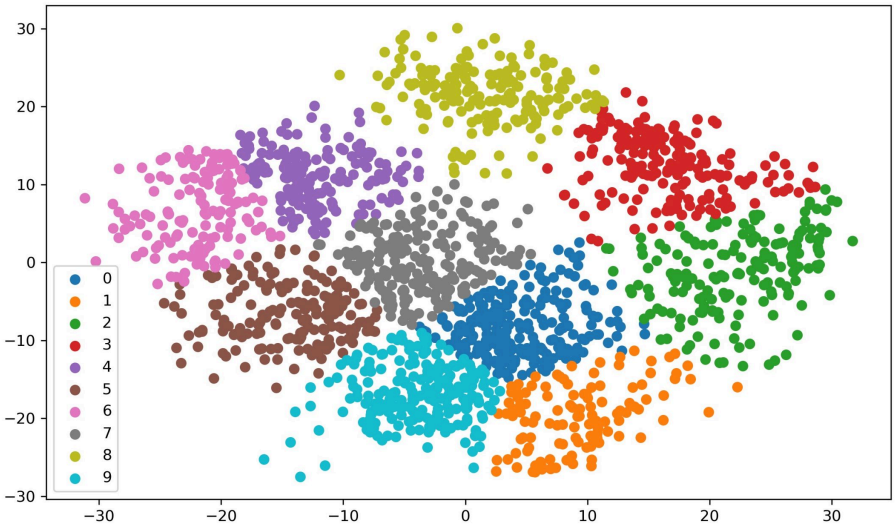

Рис. 1. Пример задачи кластеризации.

Мы ещё вернёмся к вопросу о том, что это вообще за данные и по какому признаку они разбивались, здесь важно понять саму идею: по тому или иному признаку объекты разделились на те самые кластеры.

Более строго говоря, нам необходимо построить $a : \mathbb{X} \to \{0, 1, ..., K\}$, где $a(x_i) = a(x_j) \Leftrightarrow x_i \approx x_j, i \neq j$. Возникает пару резонных вопросов:

1. Что такое K? Откуда его брать и как найти оптимальный?
2. Что означает $x_i \approx x_j$? Когда мы говорим, что объекты действительно похожи?

Метрики качества

Отвечая на второй вопрос, давайте рассмотрим некоторые основные метрики, которые в своей сущности максимально интуитивны.

Первое предположение: предполагаем, что объекты внутри одного кластера находится максимально близко центру этого кластера c_k. А значит, можно минимизировать среднее расстояние каждого объекта до соответствующего центра:[2]

$$\frac{1}{n}\sum_{k=1}^{K}\sum_{i=1}^{\ell}[a(x_i) == k]\rho(x_i, c_k) \to \min \tag{1}$$

где ρ — выбранная мера расстояния (евклидова, Манхеттена…)

Уравнение 1 называется **внутрикластерным расстоянием**.

Далее предположим следующее: мы хотим, чтобы расстояние между объектами из разных кластеров должно быть как можно больше. Запишем это:

$$\sum_{i,j=1}^{\ell}[a(x_i) \neq a(x_j)]\rho(x_i, x_j) \to \max, i \neq j \tag{2}$$

Уравнение 2 называется **межкластерным расстоянием**.

> **Замечание**
>
> Здесь достаточно легко запутаться, однако также легко запомнить интуицию: объекты в рамках одного кластера должны лежать как можно ближе к центру, но при этом объекты из разных кластеров должны лежать как можно дальше друг от друга.

Алгоритм метрической кластеризации.

Теперь, когда мы определелились с метриками, на которые возможно опираться, самое время перейти к реализации построения $a(x)$.

В рамках лекции мы рассмотрим только один алгоритм: алгоритм **K средних** (обычно называют **K-Means**). Алгоритм:

1. Случайным образом генерируем K центров: $c_1, c_2, ..., c_K$;
2. Фиксируем центры, ищем оптимальное расположение объектов по кластерам согласно Уравнение 1;
3. Теперь фиксируем сами объекты, и относительно их ищем оптимальное расположение центров согласно той же Уравнение 1[3]
4. Повторяем 2 и 3 до сходимости.

> **Замечание**
>
> Сходимостью будем считать ситуацию, при которой координаты центров кластеров изменяются незначительно (меньше какого-то установленного порога).

К слову, в случае если в качестве ρ используется евклидова метрика (должна быть ещё знакома как L_2 норма), то оптимальным решением является:

$$c_k = \frac{1}{\sum_{i=1}^{\ell}[a(x_i) == k]}\sum_{i=1}^{\ell}[a(x_i) == k]x_i \tag{3}$$

что является средним объектом с точки зрения координат (признаков).

[2] Напоминание: квадратные скобки — это индикатор события: если внутри истинна, то 1, иначе 0. В текущем примере таким образом мы рассматриваем расстояния только тех объектов, которые лежат в кластере k.

[3] Разница в том, что в шаге 2. мы ищем оптимальное решение относительно центров, а в шаге 3. — относительно объектов.

В общем и целом, K-Means справляется хорошо в ситуациях, когда для измерения расстояния между объектами уместна евклидова метрика. Для примера, алгоритм физически не сможет построить следующее разбиение:

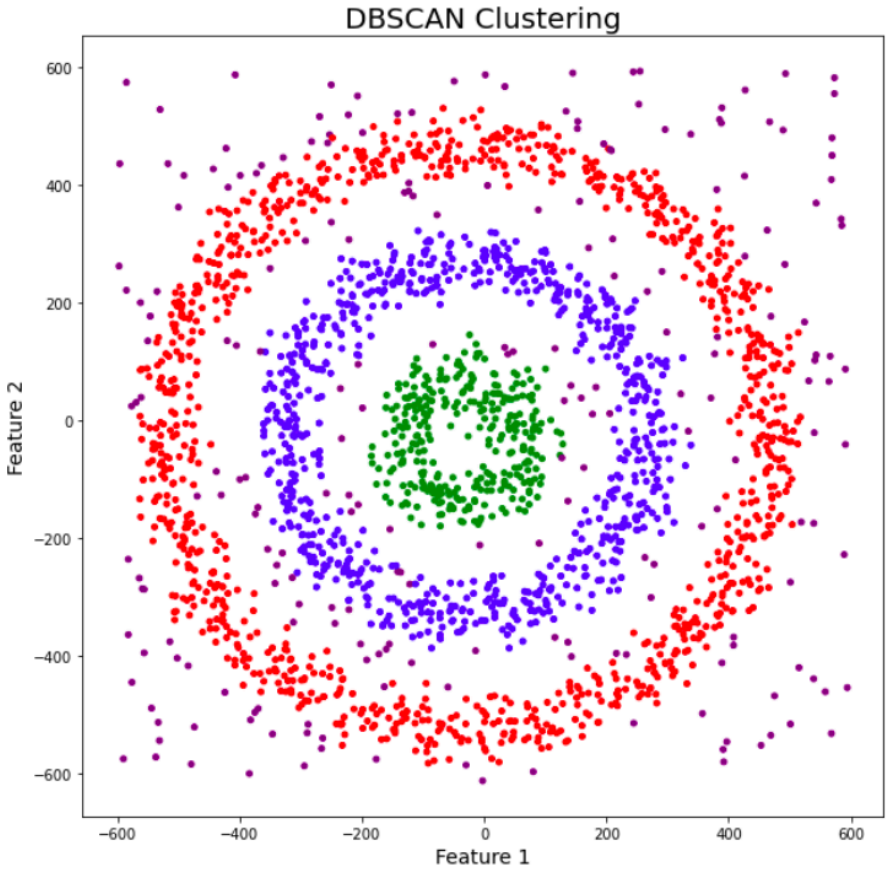

Рис. 2. DBSCAN — другой алгоритм кластеризации, который мы сегодня не обсуждаем.

Какие ещё бывают алгоритмы?

В рамках курса мы их затрагивать не будем, но тем не менее, они достойны упоминания:

- **Плотностные алгоритмы**: анализируют не расстояние, а плотность объектов в конкретных областях, что позволяет делать то, что изображено выше;
- **Иерархические алгоритмы**: строят не просто разбиение, а целое дерево вложенных кластеров, позволяя выбирать уровень детализации — от одного гигантского кластера до множества мелких групп;
- **Графовые алгоритмы**: представляют данные как сеть узлов, соединенных ребрами; это позволяет свести задачу к поиску плотных сообществ или компонент связности, что идеально подходит для данных со сложной геометрией, где обычные метрики расстояния бессильны.

Понижение размерности (Dimensionality reduction)

Определение 3 (Dimensionality reduction) Задача понижения размерности состоит в том, чтобы перевести исходную выборку $x \in \mathbb{R}^d$ в $\mathbb{R}^k$, причём $d \ll k$[4]. То есть, нужно построить функцию $a : \mathbb{R}^d \to \mathbb{R}^k$, которая при этом как можно лучше сохраняет информативность исходных признаков.

Задача возникает из предположения, что в некоторых случаях признаков уж слишком много, и нужно выделить какое-то подпространство меньшей размерности. Или же когда нам в целом нужно представить наши данные в пространстве, которое сильно меньше исходного. Это полезно для:

- Визуализации;
- Сокращения стоимости вычислений;
- Возможно даже для повышения качества, т.к. в данных может стать меньше шума / бесполезных признаков.

Например, изначально на Рис. 1 изображен датасет MNIST[5], который состоит из картинок 28×28 пикселей. Выходит, что для визуализации мы сделали

$$a : \mathbb{R}^{28\times 28} = \mathbb{R}^{784} \to \mathbb{R}^2 \tag{4}$$

и, как выяснилось, картинки даже «разбились» на какие-то осмысленные группы, которые далее мы выделили уже ранее описанной кластеризацией.

Метод главных компонент (Principal Component Analysis)

В рамках сегодняшней лекции мы рассмотрим один из основных методов понижения размерности — **Principal Component Analysis** (**PCA**). Идейно он заключается в том, что любую выборку можно представить «облаком» в форме эллипса, а уже внутри него можно выделить базис меньшей размерности, который описывает большую часть этих данных.

Рассмотрим подробнее алгоритм PCA:

1. Для начала необходимо стандартизировать все признаки, чтобы привести все признаки к одному масштабу: $x_{new} = \frac{x-a}{\sigma}$;
2. Строим ковариационную матрицу между признаками.[6] Так мы узнаем, как признаки влияют друг на друга;
3. Сделали мы это для того, чтобы выявить признаки, которые сильно зависят друг от друга (обладают высокой ковариацией). Согласитесь, что такие признаки, которые просто повторяют друг друга, являются избыточными, и точно ничего хорошего с предсказательной способностью модели не сотворят. Скорее наоборот, такая форма шума в данных негативно сказывается на модели;
4. Далее мы ищем собственные вектора и значения матрицы. Напомним, что в уравнении $Ax = \lambda x$ (т.е. при умножении матрицы на вектор, может быть, изменится только его длина, но направление останется тем же), x является собственным вектором, а λ — собственным значением;
5. Ковариационная матрица описывает, как данные «размазаны» в пространстве. Найденные собственные векторы и собственные значения интерпретируются как направления, вдоль которых данные варьируются сильнее всего. Лучше всего эта идея отображена на Рис. 3;
6. Полученные векторы будут являться «новыми признаками», которых сильно меньше, чем исходных, но которые с некоторой потерей информации будут отображать те же данные. Их и сохраняем, алгоритм закончен.

[4] $\ll$ — сильно меньше.

[5] https://en.wikipedia.org/wiki/MNIST_database

[6] В мат. статистике ковариация — характеристика между двумя случайными величинами (в нашем случае данными), которая описывает зависимость между этими величинами. Подробнее тут.

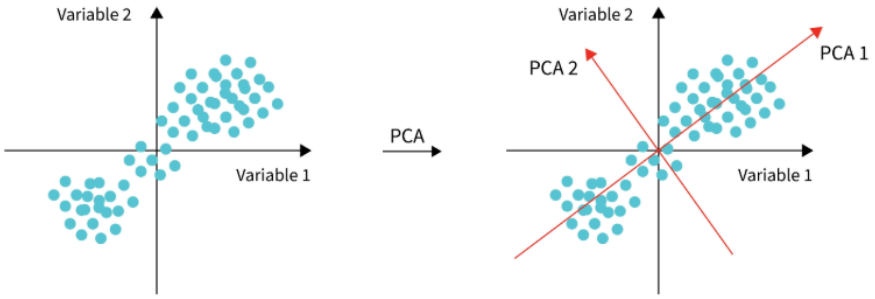

Рис. 3. Пример собственных векторов в PCA.

Естественно есть нюанс, который косвенно упоминался в алгоритме: в зависимости от разницы между размерностями, алгоритм может терять какую-то информацию. В целом, алгоритм устроен так, чтобы сохранить как можно больше информативности, но не гарантирует, что будет сохранена вся природа данных.

> **Замечание**
>
> Ещё одно небольшое замечание: новая размерность (количество собственных векторов) задаётся извне, как и в случае с количеством кластеров в K-Means. Однако если в K-Means были какие-то формальные эвристики, чтобы подобрать оптимальное значение, то в случае с понижением размерности это опускается, и желаемая размерность задаётся по усмотрению. Однако обычно принято считать, что чем меньше желаемое значение, тем больше данных мы потеряем.

Какие ещё бывают методы?

На лекции упоминались и другие методы понижения размерности, которые в своей сути сложнее, но при этом и показывают результаты зачастую лучше.

Можно упомнять **t-SNE**, смысл которого заключается в построении распределения в исходном пространстве, распределения в нужном пространстве, и минимизации расстояния Кульбака-Лейблера.[7] Не переживайте, это мы не обсуждаем.

Также отдельно стоит упомнять про **автокодировщики** (autoencoders) — это архитектура нейронной сети, которая также учится сжимать данные в пространство ниже. Однако у этого метода есть множество вариаций, один из самых популярных — **вариационный**, который после всех вышеописанных шагов учится отображать данные обратно, т.е. $\mathbb{R}^d \to \mathbb{R}^k \to \mathbb{R}^d, k \ll d$. В последствии, если убрать последний шаг, с помощью этой архитектуры можно генерировать новые данные, похожие на те, на которых он обучался. Это ещё одно из применений задачи понижения размерности, хоть и косвенных.

[7] Подробнее тут. Вы всё правильно поняли, у ML есть фандом-вики.